# Solusi Pangkalan Data Kebolehseediaan Tinggi Enterprise: MariaDB Galera + MaxScale lwn. PostgreSQL HA + Pgpool-II

> **Tajuk Dokumen:** Cetak Biru Reka Bentuk Teknikal & Penyebaran Perbandingan: Kluster MariaDB Galera + MaxScale lwn. Kebolehseediaan Tinggi (HA) PostgreSQL (Penyalinan Penyiaran Fizikal Tempatan) + Pgpool-II **Versi Dokumen:** 3.0 **Audiens Sasaran:** Arkitek Sistem, Pentadbir Pangkalan Data (DBA), Jurutera DevOps, Ketua Infrastruktur **Unit Kompetensi NOSS:** CU03 (Pentadbiran & Perkhidmatan Pelayan Linux) — WA05 (Pelaksanaan Peranan dan Servis Pelayan)

---

## 1. Ringkasan Eksekutif

Dokumen ini menyajikan cetak biru teknikal komprehensif bagi dua seni bina pangkalan data sumber terbuka gred enterprise berpengecasan Kebolehseediaan Tinggi (High Availability - HA) yang memanfaatkan proksi sedar-SQL Lapisan 7 (Layer 7):

1. **Kluster MariaDB Galera bersama MaxScale** (Proksi pangkalan data Lapisan 7 sedar-SQL).
2. **Kebolehseediaan Tinggi PostgreSQL (Penyalinan Penyiaran Fizikal Tempatan / Native Physical Streaming Replication)** bersama **Pgpool-II** (Proksi pangkalan data Lapisan 7 sedar-SQL, pengumpul sambungan / connection pooler, dan pengatur kegagalan beralih / failover orchestrator dengan Watchdog HA).

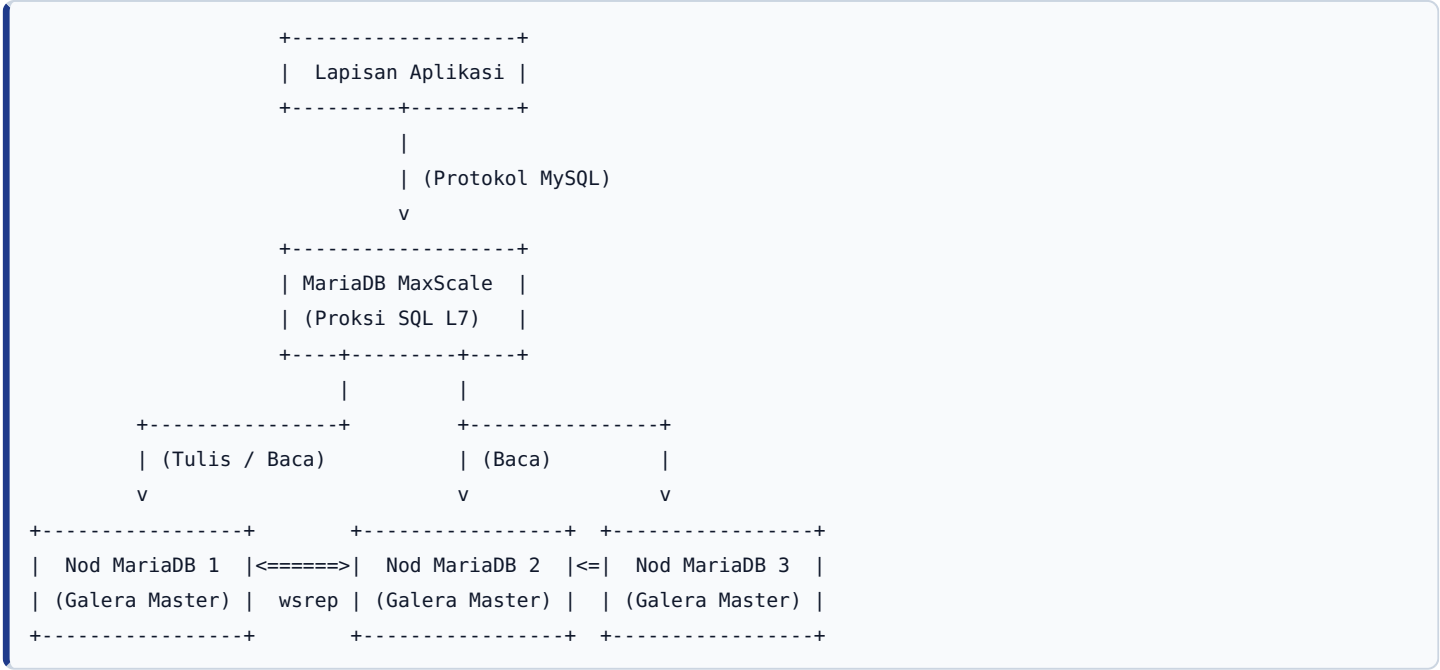
Kedua-dua seni bina menyediakan toleransi kelemahan (fault tolerance), keupayaan kegagalan beralih automatik dengan gangguan masa terhad (bounded failover interruption RTO), kebolehskalaan bacaan, pengumpulan sambungan, serta pemisahan baca-tulis telus pada satu port pangkalan data tunggal. Apabila nod Utama (Primary) mengalami kegagalan, sesi aktif yang sedang berjalan mungkin terganggu secara singkat sebelum proksi mengalihkan pautan ke nod baharu; gangguan ini terikat kepada ambang semakan kesihatan (*health check timeouts*) dan masa eksekusi skrip pemajuan (*promotion script RTO*). Reka bentuk ini memanfaatkan Pgpool-II untuk mengurus sambungan pelanggan PostgreSQL, penghalaan pertanyaan, pemantauan kesihatan, dan pemajuan nod tanpa memerlukan enjin konsensus pihak ketiga seperti `etcd` atau proksi TCP berasingan seperti HAProxy.

---

## 2. Seni Bina 1: Kluster MariaDB Galera bersama MaxScale

### 2.1 Gambaran Keseluruhan Seni Bina

Kluster MariaDB Galera ialah kluster penyalinan berasaskan multi-master dan terseganti (synchronous replication) berasaskan API *Write-Set Replication* (WSREP). MaxScale bertindak sebagai proksi pintar Lapisan 7 yang sedar-SQL untuk mengagihkan trafik berdasarkan konteks pertanyaan (memisahkan operasi baca dan tulis), melaksanakan semakan kesihatan pelayan, dan mengendalikan kegagalan beralih secara telus.



### 2.2 Komponen Teras & Tanggungjawab

- **Pelayan MariaDB (bersama Pustaka Galera):** Menyediakan penyalinan terseganti multi-master, memastikan kesemua nod mengandungi keadaan data yang identikal sebaik sahaja komit transaksi berlaku.
- **Enjin Galera WSREP:** Mengendalikan penyalinan berasaskan pensijilan (certification-based replication), komunikasi kumpulan (melalui protokol EVS), serta pemindahan keadaan data (SST/IST).
- **MariaDB MaxScale:**
- **Modul ReadWriteSplit:** Menganalisis penyataan SQL yang masuk, menghantar arahan `INSERT` / `UPDATE` / `DELETE` dan transaksi eksplisit ke sasaran penulisan utama sambil mengimbangkan beban pertanyaan `SELECT` merentas semua nod bacaan yang sedia ada.
- **Pemantau Galera (`galeramon`):** Memantau topologi kluster, status penyelarasan nod (`wsrep_local_state_comment`), dan status komponen utama secara berterusan.

### 2.3 Logik Penghalaan Trafik & Kegagalan Beralih (Failover)

- **Komit Terseganti (Synchronous Commit):** Transaksi disahkan berasaskan pensijilan di kesemua nod kluster sebelum proses komit diselesaikan.
- **Pemisahan Baca-Tulis:** MaxScale menganalisis sintaks SQL. Transaksi eksplisit (`BEGIN...COMMIT`) dan penyataan penulisan diarahkan secara khusus ke satu nod penulisan utama yang ditetapkan (bagi mengelakkan kebuntuan Galera / konflik pensijilan di bawah beban konkurensi tinggi).
- **Kegagalan Beralih (Failover):** Jika nod penulisan utama mengalami kegagalan, MaxScale mengalihkan pertanyaan penulisan secara automatik ke nod Galera lain yang terselaras. Sesi transaksi aktif semasa kegagalan mungkin terganggu dan perlu diulang oleh aplikasi, tetapi penghalaan pertanyaan penulisan baharu dipulihkan secara automatik dengan RTO $< 5$ saat.

### 2.4 Konfigurasi Garis Panduan Asas

#### A. Konfigurasi MariaDB Galera (`/etc/my.cnf.d/galera.cnf`)

```
[mysqld]
binlog_format=ROW
default_storage_engine=InnoDB
innodb_autoinc_lock_mode=2

# Tetapan asas ketahanan data ACID (1 = flushed on commit untuk integriti terjamin).
# Nota Prestasi: Nilai 0 atau 2 meningkatkan prestasi I/O penulisan tetapi mempunyai
# risiko kehilangan data sehingga 1 saat sekiranya berlaku kegagalan bekalan kuasa / kegagalan OS.
innodb_flush_log_at_trx_commit=1

# WSREP Provider Settings
wsrep_on=ON
wsrep_provider=/usr/lib64/galera-4/libgalera_smm.so
wsrep_cluster_name="mariadb_ha_cluster"
wsrep_cluster_address="gcomm://10.0.1.11,10.0.1.12,10.0.1.13"

# Node Configuration
wsrep_node_address="10.0.1.11"
wsrep_node_name="db-node-01"
wsrep_sst_method=mariabackup
wsrep_sst_auth="sstuser:<SECURE_SST_PASSWORD>"
```

**B. Konfigurasi MaxScale ( `/etc/maxscale.cnf` )**

```
[maxscale]
threads=auto

# Server Definitions
[db-node-1]
type=server
address=10.0.1.11
port=3306
protocol=MariaDBBackend

[db-node-2]
type=server
address=10.0.1.12
port=3306
protocol=MariaDBBackend

[db-node-3]
type=server
address=10.0.1.13
port=3306
protocol=MariaDBBackend

# Galera Monitor
[Galera-Monitor]
type=monitor
module=galeramon
servers=db-node-1, db-node-2, db-node-3
user=maxscale
password=<MAXSCALE_PASSWORD>
monitor_interval=2000ms
disable_master_failback=1

# Read-Write Split Router Service
[Read-Write-Service]
type=service
router=readwritesplit
servers=db-node-1, db-node-2, db-node-3
user=maxscale
password=<MAXSCALE_PASSWORD>

# Listener
[Read-Write-Listener]
type=listener
service=Read-Write-Service
protocol=MariaDBClient
port=3306
```

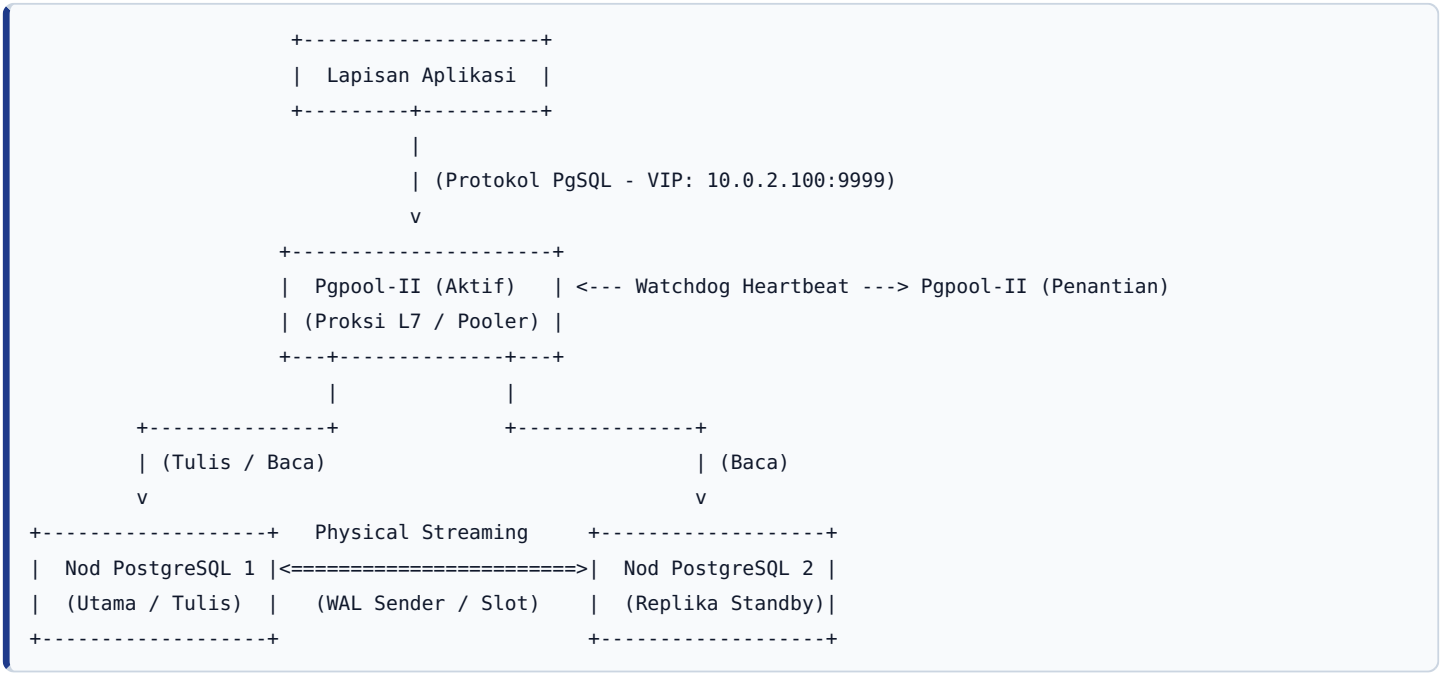
## 3. Seni Bina 2: Kebolehseediaan Tinggi PostgreSQL (Penyalinan Penyiaran Tempatan) bersama Pgpool-II

### 3.1 Gambaran Keseluruhan Seni Bina

Seni bina ini mengguna pakai Penyalinan Penyiaran Fizikal Tempatan (Physical Streaming Replication - Utama-Penantian / Primary-Standby) PostgreSQL yang diintegrasikan bersama Pgpool-II sebagai proksi Lapisan 7 sedar-SQL. Dalam Pgpool-II 4.2+, mod kluster

dikonfigurasikan secara seragam menerusi `backend_clustering_mode = 'streaming_replication'`, menyediakan pengumpulan sambungan pelanggan, pemisahan pertanyaan baca-tulis automatik pada port hadapan tunggal (9999 atau 5432), semakan kesihatan automatik, dan pengatur kegagalan beralih kluster aktif tanpa memerlukan mod lapuk *master-slave*.

Bagi menghapuskan titik kegagalan tunggal (SPOF) pada lapisan proksi, nod-nod Pgpool-II menjalankan protokol Watchdog, mengekalkan IP Maya (Virtual IP - VIP) yang dikongsi merentas berbilang instans Pgpool-II.




### 3.2 Komponen Teras & Tanggungjawab

- **Enjin PostgreSQL:** Menjalankan Penyalinan Penyiaran Fizikal tempatan (menggunakan slot penyalinan utama dan penyalinan WAL).
- **Pelayan Proksi Pgpool-II:**
- **Penganalisis Pertanyaan L7 & Pengimbang Beban:** Memintas pertanyaan PostgreSQL. Menghala penyataan `INSERT`, `UPDATE`, `DELETE`, DDL, dan transaksi eksplisit (`BEGIN...COMMIT`) ke bahagian belakang Primary PostgreSQL. Mengimbangkan beban pertanyaan `SELECT` baca sahaja merentas kesemua nod bahagian belakang yang sihat.
- **Pengumpulan Sambungan (Connection Pooling):** Mengekalkan kolam sambungan berterusan di sebelah pelayan untuk mengelakkan beban lebihan sambungan PostgreSQL.
- **Semakan Kesihatan & Kegagalan Beralih Automatik:** Memantau nod bahagian belakang secara berterusan. Mencetuskan skrip kegagalan beralih tersuai (`failover.sh`) apabila nod Utama mengalami kegagalan untuk memajukan nod pangkalan data penantian (Standby).
- **Pgpool Watchdog:** Menyelaras keadaan nod dalaman merentas berbilang instans Pgpool-II dan mengurus IP Maya (VIP) terapung menggunakan mekanisme `arping` / `keepalived`.

### 3.3 Logik Penghalaan Trafik & Kegagalan Beralih (Failover)

- **Titik Hujung Aplikasi Teragih:** Aplikasi terhubung ke Pgpool-II melalui satu port tunggal (contohnya, 9999 atau 5432).
- **Pemprosesan Pertanyaan:**
- Penyataan penulisan $\rightarrow$ Diarah terus ke Backend 0 (Utama / Primary).
- Penyataan baca sahaja $\rightarrow$ Diagihkan antara Backend 0 (Utama) dan Backend 1/2 (Replika Penantian) mengikut pemberat beban yang dikonfigurasikan.
- **Kitaran Kegagalan Beralih Automatik:**
- Jika Pgpool-II mengesan nod Utama tidak dapat dicapai (selepas melebihi `health_check_max_retries`), ia mengeksekusi `failover_command`.
- Skrip terhubung ke nod Penantian (Standby) yang ditetapkan dan mengeksekusi `pg_ctl promote` (atau mencipta fail pemicu).
- Pgpool-II mengemas kini jadual status nod dalamannya, menandakan nod Utama lama sebagai terhenti (*down*), memajukan nod Penantian kepada status Utama dalam peta bahagian belakang Pgpool, dan mengeksekusi `follow_primary_command` untuk melaras dan menyelaraskan semula nod-nod penantian lain yang tinggal.
- Gangguan pautan penulisan semasa fasa pengesahan kegagalan dan pemajuan nod terikat secara terbilang (RTO sekitar 10–15 saat), di mana pautan penulisan dipulihkan sebaik sahaja nod Penantian berjaya dipromosikan.

### 3.4 Konfigurasi Garis Panduan Asas

#### A. Konfigurasi Utama PostgreSQL ( `postgresql.conf` )

```
# Network & Connections
listen_addresses = '*'
port = 5432
max_connections = 250
shared_buffers = 4GB

# Replication Parameters & Prasyarat pg_rewind
wal_level = replica
wal_log_hints = on
full_page_writes = on
max_wal_senders = 10
max_replication_slots = 10
hot_standby = on
hot_standby_feedback = on

# Archive Parameters
archive_mode = on
archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f && cp %p /var/lib/postgresql/wal_archive/%f'
```

**B. Konfigurasi Pgpool-II (`/etc/pgpool-II/pgpool.conf`)**

```
# Network Settings
listen_addresses = '*'
port = 9999
pcp_port = 9898

# Operating Mode (Pgpool-II 4.2+)
backend_clustering_mode = 'streaming_replication'

# Backend Database Nodes
backend_hostname0 = '10.0.2.11'
backend_port0 = 5432
backend_weight0 = 1
backend_flag0 = 'ALLOW_TO_FAILOVER'

backend_hostname1 = '10.0.2.12'
backend_port1 = 5432
backend_weight1 = 1
backend_flag1 = 'ALLOW_TO_FAILOVER'

backend_hostname2 = '10.0.2.13'
backend_port2 = 5432
backend_weight2 = 1
backend_flag2 = 'ALLOW_TO_FAILOVER'

# Connection Pooling Settings
num_init_children = 32
max_pool = 4
connection_cache = on

# Load Balancing (Read-Write Splitting)
load_balance_mode = on
ignore_leading_white_space = on
white_function_list = ''
black_function_list = 'nextval,currval,lastval,setval'

# Health Check Settings
health_check_period = 5
health_check_timeout = 5
health_check_max_retries = 3
health_check_user = 'pgpool'
health_check_password = '<PGPOOL_PASSWORD>'
health_check_database = 'postgres'

# Failover & Recovery Script Commands (Bind parameter khusus nod: %D direktori gagal, %R direktori master baharu)
failover_command = '/etc/pgpool-II/failover.sh %d %h %p %D %m %H %M %P %R'
follow_primary_command = '/etc/pgpool-II/follow_primary.sh %d %h %p %D %m %H %M %P %R'

# Watchdog Settings (High Availability for Pgpool itself - Minimum 3 Nodes for Quorum in Pgpool-II 4.2+)
# Nota: Nilai ID nod tempatan (0, 1, atau 2) ditetapkan menerusi fail /etc/pgpool-II/pgpool_node_id pada setiap hos.
use_watchdog = on
wd_authkey = '<WATCHDOG_SECRET_KEY>'
delegate_IP = '10.0.2.100'
wd_lifecheck_method = 'heartbeat'
wd_interval = 2

# Node 0
hostname0 = '10.0.2.21'
wd_port0 = 9000
```

```
pgpool_port0 = 9999
heartbeat_hostname0 = '10.0.2.21'
heartbeat_port0 = 9694

# Node 1
hostname1 = '10.0.2.22'
wd_port1 = 9000
pgpool_port1 = 9999
heartbeat_hostname1 = '10.0.2.22'
heartbeat_port1 = 9694

# Node 2
hostname2 = '10.0.2.23'
wd_port2 = 9000
pgpool_port2 = 9999
heartbeat_hostname2 = '10.0.2.23'
heartbeat_port2 = 9694
```

**C. Skrip Failover Pgpool ( `/etc/pgpool-II/failover.sh` )**

Dikeksekusi secara automatik oleh Pgpool-II apabila nod bahagian belakang mengalami kegagalan. *Nota Keselamatan & Laluan:* Kunci hos pelayan (*server host-key pins*) hendaklah disimpan di dalam `~postgres/.ssh/known_hosts` pada hos Pgpool-II. Kunci awam pelanggan yang terhad (*restricted client public key*) bersama pilihan arahan dipaksa (*forced-command option*) dipasang di dalam `~postgres/.ssh/authorized_keys` pada akaun pelayan sasaran. Laluan `PGHOME` dan `PGDATA` dibina menggunakan pemboleh ubah persekitaran boleh-laras bagi menyokong pelbagai susun atur persekitaran Linux.

```
#!/usr/bin/env bash
# Parameter yang dihantar oleh Pgpool-II:
# %d = ID nod gagal, %h = hos gagal, %p = port gagal, %D = direktori data gagal
# %m = ID nod master baharu, %H = hos master baharu, %M = ID nod master lama, %P = ID nod utama lama, %R = direktori
master baharu

FAILED_NODE_ID="$1"
FAILED_HOST="$2"
FAILED_DATA_DIR="$4"
NEW_MASTER_ID="$5"
NEW_MASTER_HOST="$6"
OLD_PRIMARY_ID="$8"
NEW_MASTER_DATA_DIR="${9:-$FAILED_DATA_DIR}"

LOGFILE="/var/log/pgpool/failover.log"
PGHOME="${PGHOME:-/usr/lib/postgresql/15}"

echo "[$(date)] Cetusan kegagalan beralih. Nod Gagal: ${FAILED_NODE_ID} (${FAILED_HOST}). Utama Lama: $
{OLD_PRIMARY_ID}" >> "$LOGFILE"

# Pengesahan laluan kanonikal mutlak PGHOME dan direktori kluster
if [[ -z "$PGHOME" || ! "$PGHOME" =~ ^/[a-zA-Z0-9/_.-]+$ || "$PGHOME" == *..* ]]; then
    echo "[$(date)] RALAT: PGHOME tidak sah: '${PGHOME}'" >> "$LOGFILE"
    exit 1
fi
if [[ -z "$FAILED_DATA_DIR" || ! "$FAILED_DATA_DIR" =~ ^/[a-zA-Z0-9/_.-]+$ || "$FAILED_DATA_DIR" == *..* ]]; then
    echo "[$(date)] RALAT: FAILED_DATA_DIR tidak sah: '${FAILED_DATA_DIR}'" >> "$LOGFILE"
    exit 1
fi
if [[ -z "$NEW_MASTER_DATA_DIR" || ! "$NEW_MASTER_DATA_DIR" =~ ^/[a-zA-Z0-9/_.-]+$ || "$NEW_MASTER_DATA_DIR" ==
*..* ]]; then
    echo "[$(date)] RALAT: NEW_MASTER_DATA_DIR tidak sah: '${NEW_MASTER_DATA_DIR}'" >> "$LOGFILE"
    exit 1
fi

# Hanya lakukan pemajuan (promotion) jika nod yang gagal merupakan nod Utama
if [ "$FAILED_NODE_ID" -eq "$OLD_PRIMARY_ID" ]; then
    echo "[$(date)] Nod Utama gagal dikesan. Memulakan pemagaran tahan-lama (durable fencing/STONITH) pada $
{FAILED_HOST}..." >> "$LOGFILE"

    # Fencing tahan-lama: Hentikan servis, sekat pembukaan semula, dan bunuh proses dengan semakan pgrep ketat
    FENCE_STATUS=0
    ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"${FAILED_HOST}" \
        bash -s -- "$PGHOME" "$FAILED_DATA_DIR" << 'EOF' || FENCE_STATUS=$?
        PGHOME="$1"
        PGDATA="$2"

        # Tolak jika direktori merupakan pautan simbolik (symlink)
        if [ -L "$PGDATA" ]; then
            exit 4
        fi

        "${PGHOME}/bin/pg_ctl" stop -m immediate -D "${PGDATA}" >/dev/null 2>&1 || true
        pkill -9 -x -u postgres postgres >/dev/null 2>&1 || true

        # Bounded polling loop: semakan ketat kod status pgrep
        for ((retry=1; retry<=5; retry++)); do
            pgrep -x -u postgres postgres >/dev/null 2>&1
```

```
            RC=$?
            if [ "$RC" -eq 1 ]; then
                # Kod 1 = Tiada proses ditemui (berjaya dihentikan sepenuhnya)
                exit 0
            elif [ "$RC" -ne 0 ]; then
                # Kod >= 2 = Ralat tidak dijangka pada pgrep
                exit 2
            fi
            sleep 1
        done

        # Selepas tempoh tamat, uji sekali lagi; jika masih berjalan (RC=0), gagalkan fencing
        pgrep -x -u postgres postgres >/dev/null 2>&1
        if [ $? -eq 0 ]; then
            exit 1
        fi
        exit 0
EOF

    if [ "$FENCE_STATUS" -ne 0 ]; then
        echo "[$(date)] RALAT FATAL: Pemagaran nod (fencing) pada ${FAILED_HOST} gagal disahkan (kod: $
{FENCE_STATUS}) atau hos tidak dapat dihubungi melalui SSH! Pemagaran luar jalur (out-of-band STONITH / IPMI)
diperlukan. Menghentikan failover untuk mencegah split-brain." >> "$LOGFILE"
        exit 1
    fi
    echo "[$(date)] Pemagaran nod ${FAILED_HOST} berjaya: tiada proses PostgreSQL beroperasi." >> "$LOGFILE"

    echo "[$(date)] Memajukan nod Penantian ${NEW_MASTER_ID} (${NEW_MASTER_HOST}) kepada Utama..." >> "$LOGFILE"

    # Eksekusi arahan pemajuan melalui SSH berautentikasi kunci bukan-interaktif pada sasaran master baharu
    if ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"${NEW_MASTER_HOST}" \
        bash -s -- "$PGHOME" "$NEW_MASTER_DATA_DIR" << 'EOF'
        PGHOME="$1"
        PGDATA="$2"
        if [ -L "$PGDATA" ] || [ ! -f "${PGDATA}/PG_VERSION" ]; then
            exit 3
        fi
        exec "${PGHOME}/bin/pg_ctl" promote -D "${PGDATA}"
EOF
    then
        echo "[$(date)] Berjaya memajukan nod ${NEW_MASTER_HOST} kepada Utama." >> "$LOGFILE"
        exit 0
    else
        echo "[$(date)] RALAT: Gagal memajukan nod ${NEW_MASTER_HOST}!" >> "$LOGFILE"
        exit 1
    fi
else
    echo "[$(date)] Nod gagal ${FAILED_NODE_ID} ialah nod Penantian (Standby). Tiada pemajuan diperlukan." >>
"$LOGFILE"
    exit 0
fi
```

**D. Skrip Penyelarasan Semula Standby ( `/etc/pgpool-II/follow_primary.sh` )**

Dikeksekusi oleh Pgpool-II selepas pemajuan nod Utama baharu untuk melaras semula nod-nod penantian lain di dalam kluster 3-nod.

```
#!/usr/bin/env bash
set -e

# Parameter: %d = ID nod terjejas, %h = hos terjejas, %p = port terjejas, %D = direktori terjejas, %m = ID master
baharu, %H = hos master baharu, %M = ID master lama, %P = ID utama lama, %R = direktori master baharu
DETACHED_NODE_ID="$1"
DETACHED_HOST="$2"
DETACHED_DATA_DIR="$4"
NEW_MASTER_ID="$5"
NEW_MASTER_HOST="$6"
OLD_PRIMARY_ID="$8"
NEW_MASTER_DATA_DIR="${9:-$DETACHED_DATA_DIR}"

LOGFILE="/var/log/pgpool/follow_primary.log"
PGHOME="${PGHOME:-/usr/lib/postgresql/15}"

echo "[$(date)] Menjalankan follow_primary bagi nod ${DETACHED_NODE_ID} (${DETACHED_HOST}) menyertai $
{NEW_MASTER_HOST}..." >> "$LOGFILE"

# Pengesahan keselamatan laluan kanonikal mutlak PGHOME dan DETACHED_DATA_DIR
if [[ -z "$PGHOME" || ! "$PGHOME" =~ ^/[a-zA-Z0-9/_.-]+$ || "$PGHOME" == *..* ]]; then
    echo "[$(date)] RALAT FATAL: Laluan PGHOME tidak sah: '${PGHOME}'" >> "$LOGFILE"
    exit 1
fi

if [[ -z "$DETACHED_DATA_DIR" || ! "$DETACHED_DATA_DIR" =~ ^/[a-zA-Z0-9/_.-]+$ || "$DETACHED_DATA_DIR" == *..* || \
      "$DETACHED_DATA_DIR" == "/" || "$DETACHED_DATA_DIR" == "/var" || "$DETACHED_DATA_DIR" == "/etc" ||
"$DETACHED_DATA_DIR" == "/usr" || \
      "$DETACHED_DATA_DIR" == "/root" || "$DETACHED_DATA_DIR" == "/bin" || "$DETACHED_DATA_DIR" == "/tmp" ]]; then
    echo "[$(date)] RALAT FATAL: Laluan DETACHED_DATA_DIR tidak sah atau berbahaya: '${DETACHED_DATA_DIR}'" >>
"$LOGFILE"
    exit 1
fi

# Hentikan perkhidmatan PostgreSQL secara bersih pada nod sasaran sebelum penyelarasan semula
echo "[$(date)] Memastikan perkhidmatan PostgreSQL pada ${DETACHED_HOST} dihentikan secara bersih..." >> "$LOGFILE"
timeout 10s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"${DETACHED_HOST}"
\
    "${PGHOME}/bin/pg_ctl stop -m fast -D ${DETACHED_DATA_DIR}" >/dev/null 2>&1 || true

# Sahkan status proses PostgreSQL benar-benar terhenti menerusi pg_ctl status
STATUS_OUT=0
timeout 10s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"${DETACHED_HOST}"
\
    "${PGHOME}/bin/pg_ctl status -D ${DETACHED_DATA_DIR}" >/dev/null 2>&1 || STATUS_OUT=$?

if [ "$STATUS_OUT" -ne 3 ]; then
    echo "[$(date)] RALAT FATAL: PostgreSQL pada ${DETACHED_HOST} tidak disahkan berhenti secara selamat (kod
pulangan: ${STATUS_OUT})! Menghentikan pemulihan." >> "$LOGFILE"
    exit 1
fi

# Eksekusi pg_rewind; sekiranya gagal, guna sandaran pg_basebackup sebagai sandaran pemulihan
echo "[$(date)] Menjalankan pg_rewind pada ${DETACHED_HOST}..." >> "$LOGFILE"
if ! timeout 60s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"$
{DETACHED_HOST}" \
    "${PGHOME}/bin/pg_rewind --target-pgdata=${DETACHED_DATA_DIR} --write-recovery-conf --source-server='host=$
{NEW_MASTER_HOST} port=5432 user=postgres'"; then
```

```
    echo "[$(date)] AMARAN: pg_rewind gagal pada ${DETACHED_HOST}. Mengesahkan pemilikan dan struktur kluster
sebelum pembersihan fallback..." >> "$LOGFILE"

    # Pengesahan direktori PostgreSQL yang sah, bukan symlink, dan dimiliki postgres sebelum pembersihan destruktif
    if ! timeout 20s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"$
{DETACHED_HOST}" \
        bash -s -- "$DETACHED_DATA_DIR" << 'EOF'
        TARGET_DIR="$1"
        if [ -L "$TARGET_DIR" ]; then
            echo "Laluan merupakan symlink!"
            exit 4
        fi
        if [ ! -f "${TARGET_DIR}/PG_VERSION" ] || [ ! -d "${TARGET_DIR}/base" ] || [ ! -d "${TARGET_DIR}/global" ];
then
            echo "Direktori bukan kluster PostgreSQL yang sah!"
            exit 2
        fi
        OWNER=$(stat -c '%U' "$TARGET_DIR" 2>/dev/null || stat -f '%Su' "$TARGET_DIR" 2>/dev/null || echo "unknown")
        if [ "$OWNER" != "postgres" ]; then
            echo "Pemilikan direktori bukan postgres!"
            exit 3
        fi
        rm -rf "${TARGET_DIR:?}"/*
EOF
    then
        echo "[$(date)] RALAT: Direktori pada ${DETACHED_HOST} gagal disahkan atau pembersihan fallback dibatalkan
demi keselamatan!" >> "$LOGFILE"
        exit 1
    fi

    if ! timeout 120s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"$
{DETACHED_HOST}" \
        "${PGHOME}/bin/pg_basebackup -h ${NEW_MASTER_HOST} -p 5432 -U postgres -D ${DETACHED_DATA_DIR} -Fp -Xs -R";
then
        echo "[$(date)] RALAT: Fallback pg_basebackup juga gagal pada ${DETACHED_HOST}!" >> "$LOGFILE"
        exit 1
    fi
fi

# Mulakan semula perkhidmatan PostgreSQL sebagai Standby
if ! timeout 20s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"$
{DETACHED_HOST}" \
    "${PGHOME}/bin/pg_ctl start -D ${DETACHED_DATA_DIR}"; then
    echo "[$(date)] RALAT: Gagal memulakan semula PostgreSQL pada ${DETACHED_HOST}" >> "$LOGFILE"
    exit 1
fi

# Pengesahan kesediaan penyiaran replikasi dan had senggang (replication lag) dengan batas masa (deadline)
echo "[$(date)] Mengesahkan status penyiaran replikasi dan had latensi bagi ${DETACHED_HOST}..." >> "$LOGFILE"
READY=0
MAX_RETRIES=10
MAX_LAG_BYTES=16777216 # 16 MB had latensi dibenarkan

for ((i=1; i<=MAX_RETRIES; i++)); do
    # Bounded command checking readiness
    if timeout 5s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa postgres@"$
{DETACHED_HOST}" \
        "${PGHOME}/bin/pg_isready -h localhost -p 5432" >/dev/null 2>&1; then
```

```
        # Pertanyaan replikasi dengan had masa sambungan dan kenyataan (connection & statement timeout)
        LAG_CHECK=$(timeout 10s ssh -o ConnectTimeout=5 -o BatchMode=yes -i /var/lib/postgresql/.ssh/id_rsa
postgres@"${NEW_MASTER_HOST}" \
            "PGCONNECT_TIMEOUT=5 psql -tA -U postgres -v ON_ERROR_STOP=1 -c \"SET statement_timeout = '4000ms';
SELECT CASE WHEN state = 'streaming' AND COALESCE(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn), 0) <= $
{MAX_LAG_BYTES} THEN 1 ELSE 0 END FROM pg_stat_replication WHERE (client_addr = inet '${DETACHED_HOST}' OR
application_name = 'node_${DETACHED_NODE_ID}');\"" 2>/dev/null || echo "0")

        if [ "$LAG_CHECK" = "1" ]; then
            READY=1
            break
        fi
    fi
    echo "[$(date)] Percubaan $i/$MAX_RETRIES: Menunggu replikasi streaming dan penyelarasan WAL pada $
{DETACHED_HOST}..." >> "$LOGFILE"
    sleep 3
done

if [ "$READY" -ne 1 ]; then
    echo "[$(date)] RALAT FATAL: Nod ${DETACHED_HOST} tidak mencapai fasa streaming atau latensi replikasi melebihi
had dalam tempoh masa ditetapkan! Lampiran dibatalkan." >> "$LOGFILE"
    exit 1
fi

# Melekatkan semula nod penantian ke dalam peta Pgpool-II menerusi PCP berautentikasi selamat (~/.pcppass keizinan
0600)
pcp_attach_node -w -h localhost -p 9898 -U pgpool "${DETACHED_NODE_ID}"

echo "[$(date)] Selesai penyelarasan semula dan penempelan nod ${DETACHED_HOST} ke Pgpool-II." >> "$LOGFILE"
exit 0
```

## 4. Matriks Perbandingan Seni Bina Komprehensif

| Dimensi | Kluster MariaDB + MaxScale | PostgreSQL HA (Streaming Repl.) + Pgpool-II |
|---|---|---|
| **Jenis Penyalinan** | Terseganti / Synchronous (WSREP berasaskan pensijilan) | Penyalinan Penyiaran Fizikal Asinkronus atau Terseganti |
| **Keupayaan Multi-Master** | Aktif-Aktif (Keupayaan penulisan Multi-Master) | Aktif-Pasif (Satu Utama / Primary, Berbilang Replika Standby) |
| **Peringkat Proksi** | Lapisan 7 / Layer 7 (Sedar Protokol SQL) | Lapisan 7 / Layer 7 (Sedar Protokol SQL) |
| **Penghalaan Baca-Tulis** | Analisis penyataan SQL tempatan pada port tunggal (3306) | Analisis penyataan SQL tempatan pada port tunggal (9999 / 5432) |
| **Pengumpulan Sambungan Binaan Dalam** | Ya (Parameter `persistpoolmax` dan `persistmaxtime` ) | Ya (Pengurus kolam sambungan anak tempatan) |
| **Pengaturan Kegagalan Beralih (Failover)** | Modul `galeramon` MaxScale | Skrip eksekusi `failover_command` & `follow_primary_command` Pgpool-II |
| **Kebolehseediaan Tinggi Proksi** | Redundansi MaxScale (Keepalived / Corosync) | Protokol Watchdog Pgpool-II Tempatan bersama VIP Terapung |
| **Kebergantungan Luaran** | Tiada (Protokol WSREP binaan dalam) | Tiada (Tiada enjin DCS seperti `etcd` atau Consul diperlukan) |

| Dimensi | Kluster MariaDB + MaxScale | PostgreSQL HA (Streaming Repl.) + Pgpool-II |
|---|---|---|
| **Potensi Kehilangan Data (RPO)** | Hampir Sifar ($RPO = 0$) | $RPO > 0$ untuk replikasi asinkronus atau transaksi yang tidak menunggu nod penantian terseganti; replikasi terseganti menyokong konfigurasi keutamaan (priority) dan kuorum menerusi `synchronous_standby_names` dengan ketahanan bergantung pada `synchronous_commit` |
| **Masa Pemulihan (RTO)** | Terikat ($< 5$ saat) | Terikat (10–15 saat - Masa henti semakan kesihatan + pemajuan skrip) |
| **Risiko Kebuntuan (Deadlock)** | Sederhana jika penulisan mengenai berbilang nod serentak | Kebuntuan transaksi serentak pada nod Utama masih boleh berlaku, namun konflik penulisan silang nod dihapuskan melalui penghalaan tunggal |

---

## 5. Pertimbangan Operasi & Amalan Terbaik

### 5.1 Mencegah Senario Otak-Terpisah (Split-Brain)

- **MariaDB Galera:** Memerlukan penyebaran minimum 3 nod. Galera menggunakan pengundian kuorum berpemberat EVS; nod terpencil secara automatik memasuki keadaan bukan-utama (non-primary) bagi mengelakkan penulisan otak-terpisah.
- **PostgreSQL + Pgpool-II Watchdog:** Sebarkan bilangan nod Pgpool-II ganjil (minimum 3 instans) yang menjalankan Watchdog. Watchdog memerlukan undian majoriti ($N/2 + 1$) sebelum menetapkan IP Maya (VIP) atau mencetuskan arahan kegagalan beralih, memastikan pembahagian rangkaian tidak menghasilkan pemajuan utama otak-terpisah.

### 5.2 Strategi Sandaran & Pemulihan Bencana

- **Kluster MariaDB:** Gunakan `mariabackup` untuk sandaran fizikal tanpa sekat (*non-blocking*) yang dieksekusi pada nod bacaan penantian. Tetapkan `wsrep_desync=ON` pada nod sandaran semasa eksekusi untuk mengasingkan I/O sandaran daripada trafik penyalinan Galera.
- **PostgreSQL HA:** Sebarkan `pgBackRest` atau `WAL-G` digabungkan dengan pengarkiban WAL berterusan. Eksekusi sandaran fizikal secara terus terhadap nod pangkalan data penantian bagi mengelakkan impak CPU dan I/O cakera pada nod Utama.

### 5.3 Pengoptimuman Pengurusan Sambungan

- **MariaDB / MaxScale:** Laraskan penapis kolam sambungan MaxScale dan tetapkan had pelayan belakang ( `maxconn` ) supaya sejajar dengan parameter `max_connections` MariaDB.
- **PostgreSQL / Pgpool-II:** Penalaan parameter `num_init_children` dan `max_pool` Pgpool secara teliti. Jumlah sambungan hadapan dari Pgpool ke PostgreSQL adalah bersamaan dengan: $$\text{Jumlah Sambungan} = \text{num_init_children} \times \text{max_pool}$$ Pastikan hasil darab ini tidak melebihi parameter `max_connections` yang dikonfigurasikan pada PostgreSQL.

---

## 6. Syor & Kerangka Kerja Keputusan

### Pilih Kluster MariaDB Galera + MaxScale jika:

- Anda memerlukan seni bina Aktif-Aktif di mana mana-mana nod pangkalan data boleh menerima penulisan semasa tetingkap penyelenggaraan bergilir (*rolling maintenance*).
- Beban kerja anda memerlukan penyalinan pensijilan terseganti merentas kesemua enjin storan nod secara terus dari kotak (*out-of-the-box*).
- Anda lebih gemar enjin penghalaan SQL MaxScale yang ringan dan modular.

### Pilih PostgreSQL (Penyalinan Penyiaran) + Pgpool-II jika:

- Anda mahukan solusi Lapisan 7 serba-boleh yang menyediakan Pemisahan Baca-Tulis, Pengumpulan Sambungan, dan Kegagalan Beralih HA dalam satu lapisan proksi tunggal.
- Anda memerlukan ciri-ciri canggih PostgreSQL ( `JSONB` , `PostGIS` , sambungan/extensions) tanpa keperluan infrastruktur konsensus luaran ( `etcd` , `Consul` ).
- Anda mahukan redundansi proksi tempatan melalui Pgpool-II Watchdog tanpa perlu mengkonfigurasi perisian pengklusteran peringkat OS tambahan.

---

## 💡 Eksplorasi Lanjut bersama AI (AI Prompts)

1. *"Berikan panduan langkah demi langkah untuk menguji senario failover automatik pada Pgpool-II Watchdog menggunakan arahan pcp_watchdog_info."*
2. *"Tunjukkan contoh Playbook Ansible untuk mengautomasikan penyebaran 3-nod MariaDB Galera Cluster bersama MaxScale pada AlmaLinux 10."*
3. *"Apakah langkah penalaan prestasi kernel Linux (sysctl) terbaik untuk mengendalikan beban pangkalan data PostgreSQL berprestasi tinggi?"*

---

## 🔗 Bahan Bacaan Lanjut (Rujukan URL)

- Dokumentasi Rasmi MariaDB MaxScale
- Dokumentasi Rasmi Pgpool-II Wiki
- Panduan Penyalinan Penyiaran PostgreSQL

---

## 📚 Buku Boleh Dibeli (Syor Bacaan)

- **High Availability MySQL & MariaDB** oleh Charles Bell, Sveta Smirnova, dan Patrick Galbraith.
- **PostgreSQL High Performance** oleh Gregory Smith.
- **Panduan Praktikal Kebolehseediaan Tinggi Pelayan Linux** oleh Harisfazillah Jamel.